

Lab4 Assertions

1 Fetch

1. **Reset clears PC and instruction.** After reset is de-asserted, the PC must be 0 and the instruction output must be 0x0000.

```
assert property (  
    @(posedge clk) !rst_n | => (instr == 16'h0000 && dut.pc == 0)  
) else $error("PC_not_0_after_reset");
```

2. **Branch sets PC to branch address.** When branch_en is asserted, the PC must load branch_addr on the next cycle.

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    (branch_en) | => (dut.pc == $past(branch_addr))  
) else $error("PC_doesnt_set_to_branch_address_after_branch_en(no_reset)");
```

3. **PC increments by 2 when pc_en is active (no branch).**

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    (!branch_en && pc_en) | => (dut.pc == $past(dut.pc + 2))  
) else $error("PC_doesnt_incremented_by_2_after_pc_en(no_branch&reset)");
```

4. **PC holds when no control signal is active.**

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    (!branch_en && !pc_en) | => (dut.pc == $past(dut.pc))  
) else $error("PC_does_not_hold_its_value_when_no_control_signal_is_active");
```

5. **Instruction output matches memory at current PC.**

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    instr == dut.mem[dut.pc]  
) else $error("_instr_!=_mem[PC]");
```

6. **PC stays within range.**

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    dut.pc <= 8'hFF  
) else $error("PC_out_of_range");
```

2 Decode

1. **Decode latency:** `decode_done` asserts 2 cycles after clean acceptance. If `instr_valid` is asserted with no hazard, and no new instruction arrives at the next cycle, then `decode_done` pulses two cycles later.

```
assert property (
  @(posedge clk)
  disable iff (!rst_n)
  (instr_valid && !dut.hazard_stall_next )
  |-> ##1 !instr_valid ##1 decode_done
) else $error("decode_done_not_asserted_2_cycles_after_clean_uninterrupted_
acceptance");
```

2. **No `decode_done` during stall.**

```
assert property (
  @(posedge clk)
  disable iff (!rst_n)
  hazard_stall |-> !decode_done
) else $error("decode_done_asserted_during_stall_window");
```

3. **Field capture on acceptance.** When an instruction is accepted (valid, no hazard), the output fields must reflect the instruction word one cycle later.

```
assert property (
  @(posedge clk)
  disable iff (!rst_n)
  (instr_valid && !dut.hazard_stall_next) |=>
    (opcode == $past(instr[15:12])) &&
    (rd      == $past(instr[11:8])) &&
    (rs      == $past(instr[7:4])) &&
    (imm     == $past(instr[3:0]))
) else $error("fields_not_captured_correctly_on_acceptance");
```

4. **Field stability during hazard assertion.** When a hazard is detected, the output fields must not change.

```
assert property (
  @(posedge clk)
  disable iff (!rst_n)
  (instr_valid && dut.hazard_stall_next) |=>
    (opcode == $past(opcode)) &&
    (rd      == $past(rd)) &&
    (rs      == $past(rs)) &&
    (imm     == $past(imm))
) else $error("fields_changed_during_hazard_assertion");
```

5. **Reset clears all outputs.**

```
assert property (
  @(posedge clk)
  !rst_n |>= !decode_done && (opcode == 0) && (rd == 0) &&
    (rs == 0) && (imm == 0) && !hazard_stall
) else $error("outputs_not_cleared_after_reset");
```

6. Hazard detection: rs matches previous rd.

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    (instr_valid && (instr[7:4] == dut.last_rd)) ==> hazard_stall  
) else $error("hazard_stall_not_asserted_when_rs_matches_previous_rd");
```

7. decode_done is a single-cycle pulse.

```
assert property (  
    @(posedge clk)  
    disable iff (!rst_n)  
    decode_done ==> !decode_done  
) else $error("decode_done_asserted_for_more_than_one_cycle");
```